SAE M2 DFS & DE

Notre architecture agentique

Groupe 3 — Projet B (emplois du temps), approche agentique

Version de travail — novembre 2026

L'équipe n'a pas un outillage homogène : une partie utilise Claude Code, une autre d'autres outils. Ce document décrit d'abord ce qui est commun à tous, puis chaque configuration individuelle.

1 Socle commun à l'équipe

Élément	**Rôle**
`AGENTS.md` (versionné)	Contexte lu par l'agent à chaque session : stack, conventions de code, règles de sécurité, ce qu'il faut demander avant de faire.
Branches et PR	`feature/*` → `dev` → `main`. Tout passe par une PR relue par un humain.
Trailers de commit	`IA: genere` / `IA: retouche` / rien : origine de chaque ligne de code, pour mesurer les retouches et les bugs.
Journal `journal/ia.csv`	Une ligne par tâche : temps, interventions humaines, tokens, coût. Écrite par l'agent en fin de tâche.
Tests	pytest sur le vrai PostgreSQL. CI à venir.

2 Daniel : Claude Code

2.1 Composants

Composant	**Configuration**	**Utilisé ?**
Agent principal	Claude Code, extension VS Code	oui
Modèle	Claude Opus 5.5, effort moyen, fixé pour le projet. Sonnet 5.5 pour les questions hors code.	oui
Contexte projet	`AGENTS.md` (commun)	oui
Contexte personnel	Fichier global : langue, réponses courtes, environnement (Ubuntu, GPU), commandes à donner sur une ligne	oui
Outils web	Recherche et lecture de pages, pour vérifier les sources du rapport	oui
Skills	`claude-api` (tarifs des modèles pour le calcul du coût) ; 25 skills d'ingénierie du plugin agent-skills [1] (spec, plan, tests, revue…)	1 sur 26
Sous-agents	Intégrés : exploration, planification. Plugin : relecteur de code, auditeur sécurité, ingénieur test, auditeur performance web [2]	non
Hooks	Un hook personnel au démarrage (réseau utilisé) ; aucun hook propre au projet	non (projet)
Mémoire persistante	Disponible, vide pour ce projet	non

2.2 Boucle de travail

demande en langage naturel → l'agent explore et propose → **validation ou correction humaine** → l'agent écrit le code et les tests → commit avec trailer, push de la branche → **PR relue par un humain** → fusion dans `dev` → ligne de journal

Les actions sensibles (suppression de données, ajout de dépendance, modification du socle commun) demandent une confirmation, comme le prévoit `AGENTS.md`. L'agent ne pousse jamais directement sur `main` ou `dev`.

2.3 Usage réel à ce jour

Nombre d'appels d'outils, mesuré sur tous les transcripts du projet au 5 novembre (2 tâches terminées et leur suivi) :

Bash	Édition	Écriture	Web	Lecture	Skills / sous-agents
75	54	25	25	10	1 / 0

Constat. Un agent seul, avec un bon contexte, a suffi jusqu'ici : aucun sous-agent n'a été utile. C'est cohérent avec la recommandation de commencer simple [3]. Les sous-agents seront ajoutés seulement si le journal montre un gain.

2.4 À évaluer ensuite

- **Sous-agent relecteur** avant chaque PR : est-ce qu'il réduit le taux de retouche et les bugs ?
- **Sous-agent ingénieur test** sur le backend : est-ce qu'il améliore la couverture et le taux de mutation ?
- **Hook projet** lançant les tests avant un commit : moins de corrections après coup ?
- Chaque ajout est noté `infra-agent` dans le journal, pour mesurer son coût de mise en place.

3 Configurations 2 à 5

À compléter par chaque membre : outil, modèle, contexte, ce qui est délégué à l'IA et ce qui reste manuel.

Références

[1] A. Osmani, « agent-skills: plugin Claude Code (sous-agents et skills d'ingénierie logicielle) ». [En ligne]. Disponible sur: https://github.com/addyosmani/agent-skills

[2] Anthropic, « Claude Code documentation: Subagents ». [En ligne]. Disponible sur: https://code.claude.com/docs/en/sub-agents

[3] E. Schluntz et B. Zhang, « Building effective agents ». [En ligne]. Disponible sur: https://www.anthropic.com/engineering/building-effective-agents